hashlock.

Security Audit

Defi Bull World 3rd (DeFi)

June 2025

info@hashlock.com.au

Hashlock Pty Ltd

Table of Contents

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

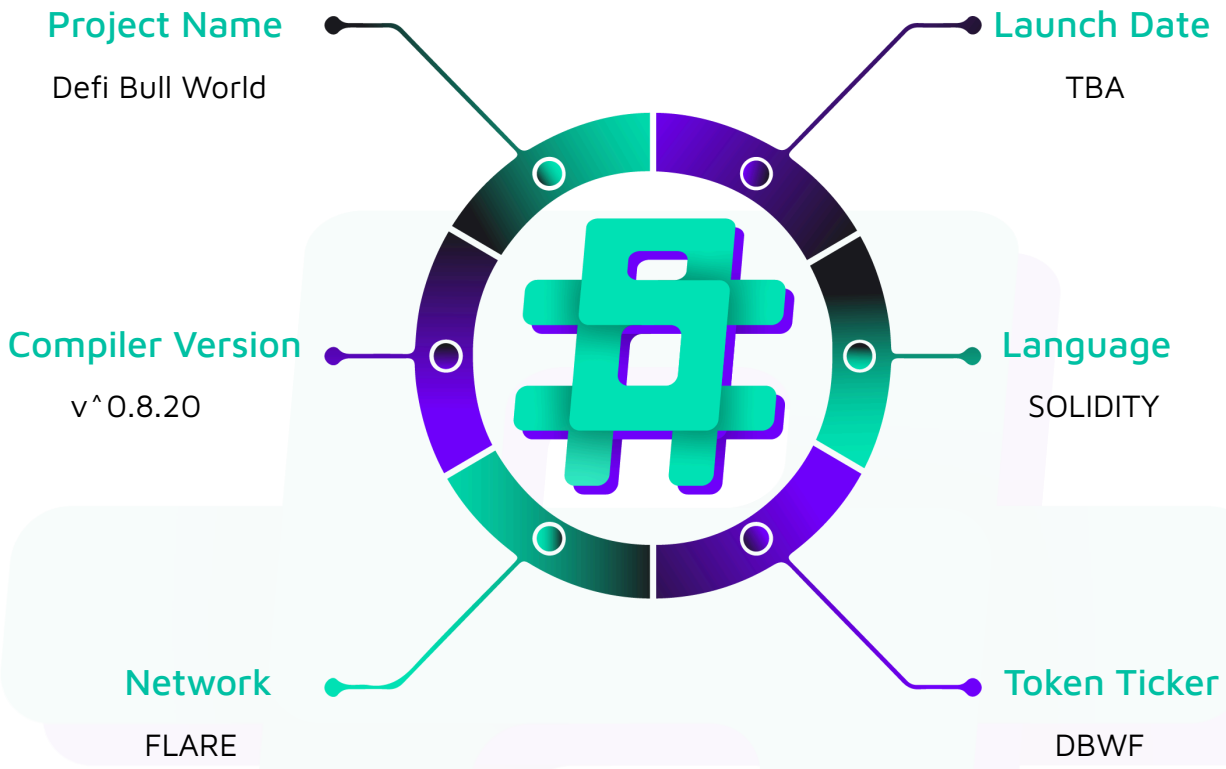
The Defi Bull World team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

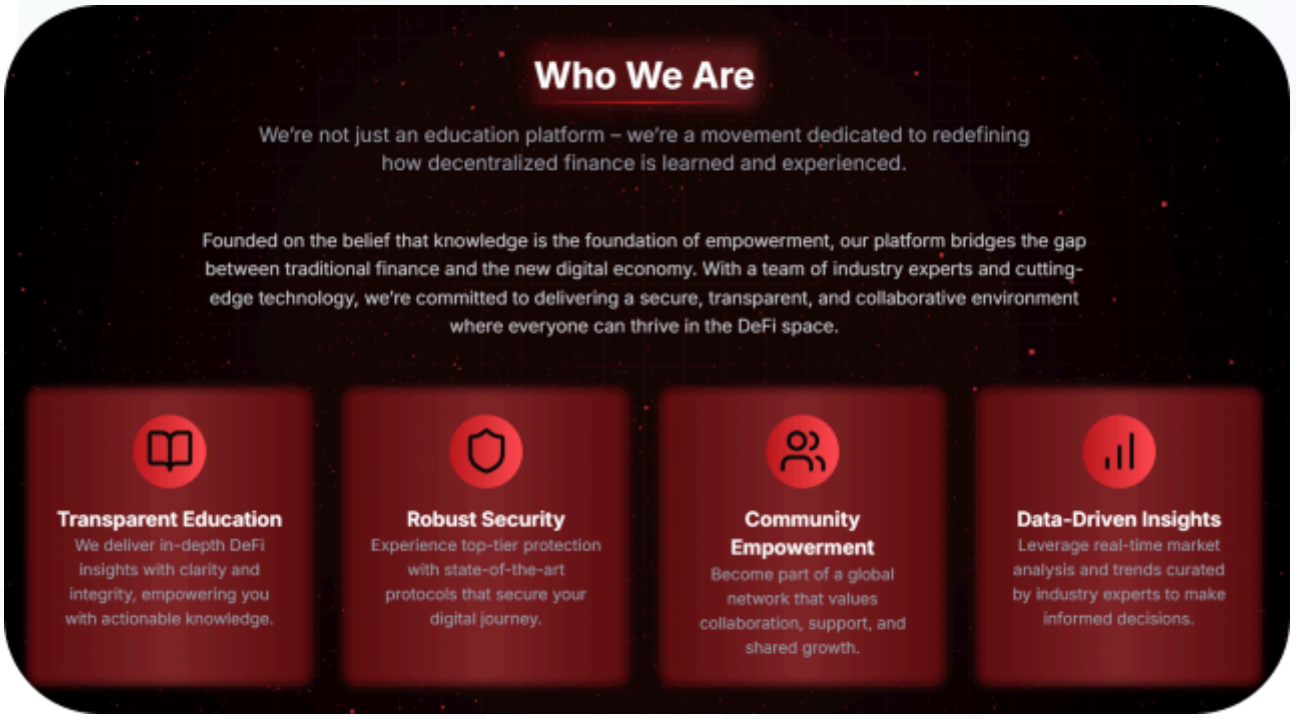
Defi Bull World is a cutting-edge digital asset and DeFi education platform built on the Flare Network with its native token FLR, led by seasoned blockchain experts since Bitcoin's inception. The platform delivers expert insights and regular updates through courses, webinars, and multimedia content, with plans to go multichain by Q3 2025. With new courses added monthly, AI-powered multilingual content, interactive experiences like sweepstakes and tests, and guaranteed long-term access for early members, Defi Bull World is committed to transparency through regular audits and public disclosure of contracts and wallets, while reinvesting revenues to drive sustainable growth.

Project Name: Defi Bull World
Project Type: DeFi
Compiler Version: ^0.8.20
Website: https://www.defibullworld.com/
Logo:

Visualised Context:



Project Visuals:



Audit Scope

We at Hashlock audited the solidity code within the Defi Bull World project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	**Defi Bull World Smart Contracts**
Platform	**Flare / Solidity**
Audit Date	**June, 2025**
Contract 1	PreSaleFixed.sol
Contract 1 MD5 Hash	c72fe9fdc6dcc723b8241d0694eea4d6

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the Audit Findings section. The list of audited assets is presented in the Audit Scope section and the project's contract functionality is presented in the Intended Smart Contract Functions section.

All vulnerabilities initially identified have now been resolved.

Hashlock found:

1 Medium severity vulnerabilities

5 Low severity vulnerabilities

1 QA

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
PreSaleFixed.sol Allows users to: - Purchase DBWF tokens with FLR (native token) - Purchase DBWF tokens with supported stablecoins (USDC, USDT, USDT0) Allows admins to: - Start the presale (sets 60-day duration) - Toggle presale active status - Update stablecoin configurations (address, decimals, name, active status) - Toggle individual stablecoin active status - Pause/unpause the contract - Add/remove allowed purchase amounts - Withdraw funds in emergency situations	**Contract achieves this functionality.**

Code Quality

This audit scope involves the smart contracts of the Defi Bull World project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Defi Bull World project smart contract code in the form of Github access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.
Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

Medium

[M-01] DBWFPresaleFixed#calculateFLRAmount - Hardcoded Price Bounds can cause DOS to FLR-Based Token Purchases

Description

This vulnerability causes a denial of service in the `DBWFPresaleFixed` contract when FLR token prices move outside hardcoded bounds ($0.001-$1.00), preventing users from purchasing tokens with FLR

Vulnerability Details

The `DBWFPresaleFixed` contract implements hardcoded price bounds validation in the `calculateFLRAmount` function that restricts FLR price acceptance to a fixed range between 100 and 100000 units (representing $0.001 to $1.00 with 5 decimal precision). The vulnerability manifests through the following code segment where oracle price data is validated against immutable constants before processing any FLR-based purchases. When the FTSO oracle reports FLR prices outside these predetermined bounds, the `require statement require(flrPriceUSD >= MIN_FLR_PRICE && flrPriceUSD <= MAX_FLR_PRICE, "Oracle price out of bounds")` will revert all transactions attempting to purchase tokens with FLR.

This validation mechanism fails to account for legitimate market volatility. The issue is critically exacerbated by the fact that these bounds are declared as public constants and the contract provides no setter function or administrative capability to modify these values during runtime. This architectural oversight means that once deployed, the contract cannot adapt to changing market conditions and users won't be able to buy tokens through FLR.

```
function calculateFLRAmount(
      uint256 tokenAmount
) public view returns (uint256) {

        (uint256 flrPriceUSD, uint256 timestamp, uint256 decimals) = ftsoOracle

            .getCurrentPriceWithDecimals();



        // Check oracle staleness - using custom error for consistency

        if (block.timestamp - timestamp > 300) revert OracleDataStale();

        // Verify decimals match expected value

             require(decimals  ==  ftsoOracle.ASSET_PRICE_USD_DECIMALS(),"Oracle  decimals
mismatch");

        // Check price bounds to prevent manipulation

        require(

            flrPriceUSD >= MIN_FLR_PRICE && flrPriceUSD <= MAX_FLR_PRICE,

            "Oracle price out of bounds"

        );

        // Use direct calculation to avoid precision loss

        uint256 usdValueInPriceTerms = (tokenAmount *

            TOKEN_PRICE_USD *

            10 ** (decimals - USD_DECIMALS)) / 10 ** 18;

        uint256 flrNeeded = (usdValueInPriceTerms * 10 ** 18) / flrPriceUSD;

      return flrNeeded;

    }
```

Impact

Exploitation of this vulnerability occurs naturally through market forces rather than malicious action, resulting in a complete denial of service for FLR-based token purchases whenever market conditions push the FLR price beyond the hardcoded boundaries.

Recommendation

The vulnerability should be addressed by implementing configurable price bounds that can be updated by authorized administrators without requiring contract upgrades. Replace the existing constant declarations with state variables such as `uint256 public minFlrPrice and uint256 public maxFlrPrice`, initialized with reasonable default values during contract deployment. Implement an administrative function `updatePriceBounds(uint256 _minPrice, uint256 _maxPrice)` restricted to the contract owner that allows dynamic adjustment of these parameters.

Status

Resolved

Low

[L-01] DBWFPresaleFixed#initialize - Contract Owner can create Inconsistent Presale State

Description

The `DBWFPresaleFixed` contract contains a critical state inconsistency issue in its presale initialization logic within the initialize function. Specifically, the contract sets `presaleActive = true` while simultaneously maintaining `presaleStarted = false`. This creates a contradictory state where the presale is marked as active but has not actually been started through the proper `startPresale()` function.

Users may experience confusion and failed transactions when attempting to purchase tokens immediately after contract deployment, as the contract appears ready for purchases but actually rejects them with PresaleNotStarted() errors.

```
function initialize(
       address _dbwfToken,
       address _ftsoOracle,
       address _usdcToken,
       address _usdtToken,
       address _usdt0Token,
       address _treasury
    ) public initializer {
        __Ownable_init(msg.sender);
        __UUPSUpgradeable_init();
        __ReentrancyGuard_init();
        __Pausable_init();
        // Presale will be started manually by owner
        presaleStartTime = 0;
```

```
        presaleEndTime = 0;

        presaleStarted = false;

        // Initialize allowed purchase amounts

        _initializeAllowedAmounts();

        // @audit this should be set as false

        presaleActive = true;

    }
```

Recommendation

The recommended fix involves aligning the initialization state with the intended contract behavior by setting `presaleActive = false` during contract initialization. This change ensures that both boolean flags accurately reflect the presale status and eliminates the contradictory state condition.

Status

Resolved

[L-02] DBWFPresaleFixed#removeAllowedAmount - Contract Owner Can Cause Unnecessary Gas Consumption Through Inefficient Array Management

Description

The vulnerability exists within the `removeAllowedAmount` function where an inefficient array element removal algorithm is implemented. When removing an element from the `allowedPurchaseAmounts` array, the current implementation performs an O(n) search to locate the target element, followed by an O(n) shifting operation to maintain array order by moving all subsequent elements one position to the left.

The problematic logic occurs in the nested loop structure where, after finding the target element at index i, the function executes a secondary loop that shifts every element from position i+1 to the end of the array one position to the left. This shifting operation requires multiple storage writes (SSTORE operations), each consuming approximately 5,000 gas units in the Ethereum Virtual Machine. For an array with n elements,

removing an element from the beginning could require up to n-1 storage writes, resulting in gas consumption of approximately (n-1) * 5,000 gas units solely for the shifting operation.

The inefficiency becomes particularly pronounced as the array grows larger.

```
function removeAllowedAmount(uint256 amount) external onlyOwner {
        if (isAmountAllowed[amount]) {
            isAmountAllowed[amount] = false;
            // Remove from array while preserving order
            uint256 length = allowedPurchaseAmounts.length;
            for (uint256 i = 0; i < length; i++) {
                if (allowedPurchaseAmounts[i] == amount) {
                    // Shift all elements after the removed element
                    // @audit more gas in this just swap low
                    for (uint256 j = i; j < length - 1; j++) {
                        allowedPurchaseAmounts[j] = allowedPurchaseAmounts[
                            j + 1
                        ];
                    }
                    allowedPurchaseAmounts.pop();
                    break;
                }
            }
            emit AllowedAmountRemoved(amount);
        }
    }
```

Recommendation

The vulnerability can be effectively remediated by implementing a swap-and-pop algorithm. This approach involves replacing the element to be removed with the last element in the array, then removing the last element using the pop() function.

```
function removeAllowedAmount(uint256 amount) external onlyOwner {
    if (isAmountAllowed[amount]) {
        isAmountAllowed[amount] = false;

        // Remove from array using swap-and-pop for gas efficiency
        uint256 length = allowedPurchaseAmounts.length;
        for (uint256 i = 0; i < length; i++) {
            if (allowedPurchaseAmounts[i] == amount) {
                // Move last element to the position of element to be removed
                // This is more gas efficient
                allowedPurchaseAmounts[i] = allowedPurchaseAmounts[length - 1];
                allowedPurchaseAmounts.pop();
                break;
            }
        }
        emit AllowedAmountRemoved(amount);
    }
}
```

Status

Resolved

[L-03] DBWFPresaleFixed - Use Ownable2Step version rather than Ownable version

Description

Ownable2StepUpgradeable prevents the contract ownership from mistakenly being transferred to an address that cannot handle it (e.g. due to a typo in the address), by

requiring that the recipient of the owner's permissions actively accept via a contract call of its own.

Recommendation

Consider using Ownable2StepUpgradeable and Ownable2Step from OpenZeppelin Contracts to enhance the security of your contract ownership management. This contract prevents the accidental transfer of ownership to an address that cannot handle it, such as due to a typo, by requiring the recipient of owner permissions to actively accept ownership via a contract call.

Status

Resolved

[L-04] DBWFPresaleFixed - Floating and Outdated Pragma version of Solidity

Description

The project uses `pragma solidity ^0.8.20`. 0.8.20 is an outdated and floating pragma version of Solidity

Recommendation

Use `pragma solidity 0.8.28`.

Status

Resolved

[L-05] DBWFPresaleFixed#initialize - Missing zero address checks

Description

The _usdcToken, _usdtToken and _usdt0Token in `initialize()` are missing zero address checks for the input parameters values.

Recommendation

Add checks if the input parameters values are zero or not.

Status

Resolved

QA

[Q-01] DBWFPresaleFixed- Unused Events

Description

Event `PresaleConfigured` has been added in the contract but never used.

Recommendation

Use the event wherever required or remove it if not required.

Status

Resolved

Centralisation

The Defi Bull World project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.

Conclusion

After Hashlock's analysis, the Defi Bull World project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au
Contact: info@hashlock.com.au